Apex — Test Plan

Project: Apex — Real-Time Data Integrity Framework for TradingView

Author: Shiv Shanker Gupta

Date: 24 Sep 2025

---

1. Project Overview (Short & Clear)

Apex is an automation framework to validate that the **visual chart** in TradingView matches the authoritative OHLC data from the Alpha Vantage API. The goal is to prove *data integrity* (historical OHLC equality), automation of complex chart interactions (indicators, timeframe changes), and basic liveness checks for real-time streams.

This test plan describes the scope, objectives, test approach, environment, test cases, schedule, risks, and reporting required to deliver a professional, interview-ready project.

---

2. Objectives (What we must prove)

1. **Historical Integrity:** For a chosen symbol (e.g., NASDAQ:AAPL), verify that TradingView's candle OHLC for a specific trading date equals Alpha Vantage OHLC (within rounding tolerance).
2. **Chart Control:** Successfully add an indicator (MACD) and verify its panel appears.
3. **Timeframe Validity:** Change timeframes (1D → 1W → 1M) and verify chart updates.
4. **Real-time Liveness:** Verify the main price ticker updates over time and conforms to numeric format.

5. **Stability & Reporting:** Provide robust Extent Reports showing API vs UI comparisons, screenshots, and CI integration via GitHub Actions.

---

3. Scope

In scope:

- Automated tests for historical OHLC verification (1 target date)
- Automated tests for the MACD indicator addition
- Automated tests for the timeframe change
- Automated real-time liveness test (price ticker)
- Extent Report generation + screenshots
- CI pipeline with GitHub Actions (headless run)

Out of scope:

- Deep statistical comparisons across long time series (bulk data validation)
- Non-UI API-only validation beyond Alpha Vantage
- Performance/load testing

---

4. Test Strategy & Approach

Hybrid testing approach: Manual verification for initial exploratory checks and automated tests for repeatable validation.

Tools & Framework:

- Java (17), Maven
- Selenium 4 (WebDriver + Actions), TestNG
- Cucumber for BDD feature(s)
- Alpha Vantage API client (Java HttpClient + Jackson)

- Extent Reports for HTML reporting
- GitHub Actions for CI

Key techniques:

- Use the `symbol` query param to load TradingView for a symbol (stable entry point).
- Use **keyboard navigation (arrow keys)** to move the crosshair between candles (more stable than pixel offsets on canvas).
- Use robust text-based XPaths that look for `O/H/L/C` labels in the tooltip to extract values.
- Handle weekends/market holidays by picking the most recent trading day ≤ requested date from the API.
- Compare numeric values with rounding to 2 decimals or epsilon tolerance (0.01).

---

5. Environment Matrix

Layer	Detail
Local Dev	Windows 10/11, Chrome (latest), Java 17, IntelliJ IDEA
CI	ubuntu-latest (GitHub Actions), headless Chrome, Java 17
Test Data	Live Alpha Vantage API (secrets: `ALPHAVANTAGE_API_KEY`)
Browsers	Chrome (primary). Firefox is optional for cross-browser later.

Environment setup checklist:

1. Set `ALPHAVANTAGE_API_KEY` as an environment variable or GitHub secret.
2. Maven build (`mvn clean test`) with headless flag `-Dheadless=true` for CI.

3. Ensure internet access in CI (to call Alpha Vantage and load TradingView).

---

6. Test Data & Dates

- **Symbols:** NASDAQ: AAPL (primary). Prepare the config to change the symbol easily.
- **Dates:** Use `requestedDate = yesterday` (but if weekend/holiday, use API's last trading day ≤ requestedDate). Log the chosen date in reports.
- **API key:** must be kept secret and injected via environment variable.

---

7. Test Cases (High priority, detailed)

TC-01: Historical OHLC Integrity — AAPL (High)

Objective: Validate TradingView candle OHLC equals Alpha Vantage. **Preconditions:** API key present. TradingView loaded with `symbol=NASDAQ: AAPL`. Timeframe = 1D. **Steps:**

1. Call AlphaVantageClient fetchDailyOhlcForDate(AAPL, requestedDate).
2. Open TradingView chart URL with symbol param.
3. Focus the chart and move the crosshair to the trading date using keyboard arrows until the tooltip date == chosenDate.
4. Read tooltip O/H/L/C values and parse as doubles.
5. Compare API vs UI values (round to 2 decimals). Log both in Extent.
6. Attach a screenshot. **Expected:** All four values match within epsilon. **Pass/Fail:** Fail if any value differs beyond tolerance or the tooltip is not found.

TC-02: MACD Indicator Add (Medium)

Objective: Add MACD and verify presence. **Steps:**

1. Click the Indicators button.

2. Type `MACD` and select `Moving Average Convergence Divergence`.
3. Wait for the MACD panel.
4. Assert panel is visible and contains the `MACD` label. **Expected:** MACD panel present.

TC-03: Timeframe Change (Medium)

Objective: Verify that switching timeframes updates the chart. **Steps:**

1. Click the timeframe selector and choose `1W`.
2. The assert header/timeframe element shows `1W` or candles visually changing.
3. Repeat for `1M`. **Expected:** Timeframe shows correctly; tooltip/candles correspond to timeframe.

TC-04: Real-time Liveness & Format (Medium)

Objective: Prove ticker updates and format correctness. **Steps:**

1. Read price at T1.
2. Wait 5 seconds.
3. Read price at T2.
4. Assert T1 != T2 and both match regex `^[0-9]+\.[0-9]{2}$` (adjust if thousands separators present). **Expected:** Values differ, and format is correct.

---

8. Test Execution Schedule (Suggested)

- **Week 1:** Environment setup, AlphaVantage client, basic Selenium skeleton, manual exploratory runs.
- **Week 2:** Implement Historical OHLC test, robust selectors, Extent reporting, screenshots.
- **Week 3:** Implement MACD & timeframe tests, liveness test, Cucumber feature, step-definitions.
- **Week 4:** CI integration (GitHub Actions), cross-check flakiness, finalize README, sample report.

9. Entry / Exit Criteria

Entry Criteria:

- Project codebase scaffolded (Maven pom, test skeleton).
- Alpha Vantage API key available.
- Chrome driver available via Selenium Manager.

Exit Criteria:

- All high-priority tests (TC-01 to TC-04) pass reliably (> 90% local stability).
- Extent report produced with API vs UI comparisons and screenshots.
- CI runs pass on the main branch (headless mode) at least once.

10. Reporting & Artifacts

Deliverables:

- Maven project in GitHub repo `Apex` with README.
- Extend HTML reports in the `reports/` directory for each run (attach sample in repo).
- Cucumber feature file(s) and step definitions.
- `.github/workflows/main.yml` for CI.
- Test the evidence folder with screenshots and logs.

Reporting cadence:

- Each test run creates an Extent report summarizing passes/fails and embedding API vs UI logs.
- For failures: capture a screenshot, page HTML snapshot, and include API response JSON in the report.

11. Risks & Mitigations

- **Risk:** TradingView DOM keys or tooltip structure change. **Mitigation:** Use text-based XPaths and fallback locators; maintain small wrapper methods for selectors to update quickly.
- **Risk:** Alpha Vantage rate limits (5 calls/min free tier). **Mitigation:** Cache API responses during a test run; avoid repeated calls; for CI, use a small output size and single-date lookup.
- **Risk:** CI network restrictions or blocked external resources. **Mitigation:** Ensure network egress is allowed; mock API for offline runs (future enhancement).
- **Risk:** Flaky navigation on canvas. **Mitigation:** Use keyboard navigation with throttled delays and max attempts + screenshot logging on failure.

12. Traceability Matrix (Key)

Requirement	Test Case ID
Historical OHLC correctness	TC-01
Add the MACD indicator	TC-02
Timeframe change correctness	TC-03
Price liveness & format	TC-04

13. Sample Test Case Template (to use in Excel/TestRail)

Test Case ID: TC-01 **Title:** Historical OHLC Integrity — AAPL **Preconditions:** API key set; internet available. **Steps:** (as TC-01 steps) **Expected Result:** API & UI OHLC match. **Actual Result:** (to fill) **Status:** Pass/Fail **Attachments:** Screenshot, API JSON

---

14. Notes for Interviewer-Ready Presentation

- Keep README crisp: purpose, architecture diagram (simple), how to run tests locally & in CI, and how to add API key.
- Include a sample Extent Report (failing and passing example) so the interviewer sees both happy and edge cases.
- Document troubleshooting tips in README: how to update selectors, how to change symbol/date, and how to re-run a single test.

---

15. Next Actions (What you can ask me to produce now)

1. I can export this test plan into a printable Word or PDF file.
2. I can generate the full Maven `pom.xml` + `AlphaVantageClient.java` + `TradingViewPage.java` + `HistoricalDataTest.java` ready to paste into your IDE.
3. I can create the Cucumber feature + step definitions and an example Extent Report template.